

MDS-002 — Colour System

External publication draft

MDS-002 — Colour System

Colour should communicate understanding before identity, and identity before decoration.

Purpose

MDS-001 established how design intent becomes machine-readable through the Design Token Architecture.

MDS-002 defines how colour expresses that intent.

Unlike conventional colour systems, the Mosaic Colour System is not centred around a brand palette.

Instead, it balances three independent responsibilities:

- Brand Identity
- Semantic Communication
- Runtime Atmosphere

These responsibilities intentionally remain separate.

The result is a colour system that is:

- recognisable
- adaptive
- accessible
- artwork-aware
- implementation independent

Relationship to Previous Specifications

Vision



Principles



Mental Model



Interaction



Composition



Design Tokens



Colour System



Material System



Components

The Colour System implements the semantic architecture established by MDS-001.

It never replaces it.

Scope

This specification defines:

- Brand Colours
- Semantic Colours
- Runtime Atmosphere
- Artwork Colour Extraction
- Adaptive Colour
- Theme Architecture
- Light Mode
- Dark Mode
- Accessibility
- Colour Resolution

This specification intentionally does **not** define:

- Materials
- Components
- Typography
- Motion
- Layout

Those specifications consume colour.

They do not define it.

Core Question

MDS-002 exists to answer one question.

How should colour communicate meaning?

Not:

Which colours should we use?

Colour Statement

Within Mosaic:

Colour communicates understanding first.

Brand second.

Decoration last.

If colour ever becomes the primary source of understanding, accessibility has already failed.

Colour Responsibilities

The Mosaic Colour System separates colour into three independent systems.

Brand

↓

Semantic

↓

Atmosphere

Each system exists for one purpose.

They intentionally do not overlap.

Expected Outcome

After reading MDS-002 contributors should understand:

- how Mosaic uses colour
- how artwork influences the interface
- how runtime atmosphere works
- how accessibility is preserved
- how themes evolve
- how clients resolve colour consistently

without discussing implementation.

Repository Structure

design/

■■■ mds/

■■■ MDS-002 Colour System/

README.md

00-document-control.md

01-colour-philosophy.md

02-brand-colours.md

03-semantic-colours.md

04-runtime-atmosphere.md

05-artwork-colour-extraction.md

06-theme-architecture.md

07-light-and-dark.md

08-accessibility.md

09-colour-resolution.md

10-runtime-synthesis.md

11-governance.md

12-adrs.md

13-contributor-guidance.md

glossary.md

references.md

Dependencies

Required reading:

- MDL-001 → MDL-005
- MDS-001 Design Token Architecture

Downstream specifications:

- MDS-003 Material System
- MDS-005 Motion System
- MDS-006 Composition Engine
- MDS-008 Component Library

Review Status

Status

Draft

Owner

Lead Design Systems Architect

Next File

00-document-control.md

Document Control

Document Information

Property	Value
Document ID	MDS-002
Title	Mosaic Design System — Colour System
Classification	Internal
Status	Draft
Version	0.1
Owner	Lead Design Systems Architect
Parent Specifications	MDL-001 → MDL-005, MDS-001
Repository	/design/mds/MDS-002 Colour System/

Purpose

MDS-002 defines the Colour System used throughout the Mosaic Design System.

Unlike conventional colour systems, which primarily establish visual identity, the Mosaic Colour System has three independent responsibilities.

1. Preserve the Mosaic brand. 2. Communicate semantic meaning. 3. Reflect the user's current entertainment atmosphere.

These responsibilities intentionally remain independent.

The Colour System exists to support understanding.

It should never compete with the entertainment it presents.

Authority

MDS-002 governs:

- Brand Palette
- Semantic Colour Mapping
- Runtime Atmosphere
- Artwork Colour Extraction
- Theme Architecture
- Colour Resolution
- Colour Accessibility
- Colour Adaptation

This specification intentionally does **not** govern:

- Materials
- Typography
- Components
- Motion
- Layout

Those systems consume colour.

They do not define it.

Relationship To MDS

The Colour System extends the Design Token Architecture.

```
[mermaid]
flowchart TD
```

```
Vision
Vision --> Principles
Principles --> MentalModel
MentalModel --> Interaction
Interaction --> Composition
Composition --> Tokens
Tokens --> Colour
Colour --> Material
Material --> Components
```

The Colour System consumes Semantic Tokens.

It never introduces new semantic meaning.

Design Intent

The Colour System intentionally separates three independent ideas.

```
[text]
Brand
```

↓

Meaning

↓

Atmosphere

This separation is fundamental.

Brand should never determine semantic meaning.

Artwork should never redefine the Mosaic brand.

Atmosphere should never weaken accessibility.

Each layer performs exactly one responsibility.

Reader Expectations

Before reading this specification contributors should already understand:

- MDL-001 Vision
- MDL-002 Principles
- MDL-003 Mental Model
- MDL-004 Interaction Model
- MDL-005 Composition Model
- MDS-001 Design Token Architecture

This document assumes the conceptual architecture has already been established.

Its responsibility is implementation.

Architectural Scope

The Colour System defines:

- Brand Colour Architecture
- Semantic Colour Roles
- Adaptive Atmosphere
- Runtime Colour Behaviour
- Accessibility Rules
- Colour Resolution

It intentionally avoids discussing implementation frameworks such as:

- CSS
- Flutter
- SwiftUI
- Compose

Those are generated from the Colour System rather than defining it.

Stability

Expected lifetime.

Artefact	Expected Lifetime
Primitive Colour Values	Years
Brand Palette	Many Years
Semantic Colour Roles	Many Years
Runtime Colour Algorithms	Occasionally

Artefact	Expected Lifetime
Presentation Themes	Frequently

Colour values may evolve.

Semantic meaning should remain significantly more stable.

Success Criteria

MDS-002 succeeds when:

- screenshots are immediately recognisable as Mosaic
- artwork enhances rather than overwhelms the interface
- semantic meaning remains consistent across themes
- accessibility remains uncompromised
- runtime adaptation feels natural rather than decorative
- colour communicates understanding before aesthetics

Users should remember:

- their entertainment

and

- the feeling of Mosaic.

They should rarely notice the Colour System consciously.

Review Status

Status

Draft

Dependencies

- MDL-001 → MDL-005
- MDS-001 Design Token Architecture

Supersedes

None.

Next File

01-colour-philosophy.md

Colour Philosophy

Purpose

Before defining palettes, themes or runtime colour systems, contributors must first understand what colour means within Mosaic.

Many Design Systems begin with colour values.

Mosaic intentionally does not.

It begins with philosophy.

Colour exists to strengthen the user's relationship with their entertainment.

It should never become the entertainment itself.

Philosophy Statement

Colour should quietly support understanding while allowing entertainment to remain the emotional centre of the experience.

Everything within the Mosaic Colour System derives from this statement.

Colour Is Communication

Within Mosaic, colour is considered a communication system.

It communicates:

- hierarchy
- identity
- status
- atmosphere
- confidence

Colour should never exist purely for decoration.

If removing a colour changes nothing about understanding...

That colour probably does not belong.

Three Responsibilities

The Mosaic Colour System intentionally separates colour into three independent responsibilities.

[text]
Brand

↓

Semantic Meaning

↓

Atmosphere

Each responsibility answers a different question.

Responsibility	Question
Brand	"Who is speaking?"
Semantic	"What does this mean?"
Atmosphere	"How does this feel?"

Keeping these responsibilities separate allows the system to evolve without conflict.

Colour Exists In Service Of Content

The most important colour within Mosaic is often **not** a Mosaic colour.

It is the artwork.

Album covers.

Film posters.

Book covers.

Anime key visuals.

These already possess emotional identity.

Mosaic should respect that identity rather than competing with it.

The interface should frame artwork.

Not overpower it.

Colour Is Secondary To Hierarchy

Hierarchy should remain understandable even if colour disappears.

Users should still understand:

- what matters
- what is secondary
- what is interactive
- what has changed

using:

- composition
- spacing
- typography

- movement

Colour reinforces understanding.

It should never become the only mechanism communicating it.

This principle ensures accessibility remains fundamental rather than optional.

Colour Is Calm

Many applications use colour aggressively.

Examples include:

- promotional gradients
- high saturation
- competing accents
- constant emphasis

Mosaic intentionally adopts a calmer philosophy.

Most interface surfaces should remain restrained.

Colour should be introduced deliberately where it improves:

- orientation
- confidence
- understanding

Silence is as valuable in colour as it is in interaction.

Brand Is Stable

The Mosaic brand should remain recognisable.

Regardless of:

- artwork
- media type
- accessibility
- runtime atmosphere

users should still recognise the application as Mosaic.

The Brand Palette therefore remains comparatively stable.

It communicates identity.

Not emotion.

Atmosphere Is Adaptive

Emotion should come from the user's current entertainment.

Examples.

Watching:

Blade Runner 2049

The interface may become subtly cooler.

Watching:

Your Name

The interface may become subtly warmer.

Reading:

The Hobbit

The interface may become subtly earthy.

Importantly...

The atmosphere should feel like reflected light.

Not a recoloured application.

This distinction is one of the defining characteristics of Mosaic.

Colour Should Never Surprise

Unexpected colour changes weaken trust.

Runtime adaptation should therefore be:

- gradual
- contextual
- understandable

Users should instinctively feel:

"The interface reflects what I'm enjoying."

Not:

"The interface changed colour."

Atmosphere should be perceived emotionally before it is noticed intellectually.

Neutral First

Most of the interface should intentionally remain neutral.

Neutral surfaces provide three advantages.

1. Artwork becomes more expressive. 2. Accessibility becomes easier. 3. Runtime atmosphere becomes more effective.

The interface should therefore spend most of its time acting as a canvas.

Not as artwork.

Accent With Purpose

Accent colours should communicate action.

Not decoration.

Examples include:

- primary actions
- current focus
- active playback
- selection
- progress

Accent should never become background.

Likewise...

Background should never compete with accent.

This preserves a strong visual hierarchy.

Colour Evolves

Colour is expected to evolve over time.

The same semantic meaning may resolve differently depending upon:

- artwork
- theme
- accessibility
- display capabilities

However...

The conceptual meaning must remain stable.

Example.

`Surface.Hero`

may resolve differently in:

- Light Mode
- Dark Mode

- Runtime Atmosphere

The semantic role remains identical.

Good Examples

Example 01

Current artwork provides subtle cyan highlights.

The interface gently reflects those tones through acrylic materials.

Brand accents remain recognisable.

The artwork remains dominant.

Example 02

Accessibility increases contrast.

Runtime atmosphere reduces intensity.

Semantic meaning remains identical.

The interface remains recognisably Mosaic.

Example 03

The Hero surface adopts a warmer reflected tone from book artwork.

Surrounding surfaces remain neutral.

Attention naturally remains on the book cover.

Anti-patterns

Artwork Recolouring Everything

Every interface element adopts artwork colours.

Brand identity disappears.

Hierarchy weakens.

Brand Everywhere

Every interface element uses the Mosaic brand colour.

Artwork becomes visually irrelevant.

Colour As Hierarchy

Removing colour destroys understanding.

The Composition has become dependent upon colour.

Decorative Gradients

Colour exists because it appears visually impressive.

No additional understanding is communicated.

Philosophical Model

[mermaid]
flowchart TD

```
Brand
Brand --> SemanticMeaning["Semantic Meaning"]
SemanticMeaning["Semantic Meaning"] --> RuntimeAtmosphere["Runtime Atmosphere"]
RuntimeAtmosphere["Runtime Atmosphere"] --> Presentation
Presentation
Presentation --> Entertainment
```

Brand establishes identity.

Semantic colour communicates meaning.

Atmosphere reflects entertainment.

The interface quietly supports the user's World.

Relationship To Future Chapters

The remaining chapters formalise this philosophy into implementation.

They define:

- Brand Palette
- Semantic Colour Architecture
- Runtime Atmosphere
- Artwork Extraction
- Theme Resolution
- Accessibility

Every implementation decision should reinforce the philosophy established here.

Summary

Colour is one of the quietest systems within Mosaic.

When successful, users should rarely think about it consciously.

Instead they should simply feel that:

- the interface belongs with their entertainment,
- the platform feels calm,
- the artwork feels more alive,
- and Mosaic always feels unmistakably like Mosaic.

That balance between identity, meaning and atmosphere is the defining objective of the Mosaic Colour System.

Review Status

Status

Draft

Next File

02-brand-colours.md

Brand Colours

Purpose

The Mosaic Brand Palette establishes the permanent visual identity of the platform.

Unlike Runtime Atmosphere, which adapts to the user's entertainment, Brand Colours should remain immediately recognisable across every client, operating system and device.

Brand Colours answer one question.

"Am I using Mosaic?"

They do **not** answer:

- what currently matters,
- what is interactive,
- what the current media feels like.

Those responsibilities belong to Semantic Colours and Runtime Atmosphere.

Philosophy

The Mosaic Brand should be immediately recognisable.

However...

It should rarely dominate the interface.

Unlike many entertainment platforms whose identity depends upon overwhelming colour saturation:

- Netflix Red
- Spotify Green
- Plex Yellow
- Jellyfin Purple

Mosaic intentionally adopts a quieter philosophy.

Its brand should be:

- memorable
- modern
- restrained
- premium

The brand should support the user's entertainment.

Not compete with it.

Brand Architecture

The Brand Palette consists of three conceptual layers.

[text]

Core Brand

↓

Supporting Brand

↓

Accent Brand

Each layer performs one responsibility.

Core Brand

Purpose.

Communicate Mosaic itself.

The Core Brand Colour appears in:

- application identity
- onboarding
- authentication
- splash screens
- installer
- branding assets

It should appear sparingly throughout the runtime interface.

The brand should remain recognisable because of restraint.

Not repetition.

Supporting Brand

Supporting Brand Colours provide visual harmony around the Core Brand.

Examples include:

- gradients
- illustrations
- documentation
- diagrams
- empty states

Supporting colours should never become primary interaction colours.

Their purpose is identity.

Not hierarchy.

Accent Brand

Accent Brand Colours provide subtle moments of recognition.

Examples include:

- loading indicators
- active installation
- progress
- onboarding confirmation
- branding moments

Accent Brand Colours should appear intentionally.

Users should associate these colours with Mosaic itself rather than entertainment content.

Brand Neutrality

Most of the runtime interface should intentionally avoid strong brand colouring.

Example.

Instead of:

Entire Interface

↓

Brand Cyan

Prefer:

Neutral Interface

↓

Small Brand Moments

This approach creates two advantages.

1.

Artwork remains emotionally dominant.

2.

Brand moments become significantly more memorable.

Brand Persistence

The Brand Palette should remain recognisable regardless of:

- Light Mode
- Dark Mode
- Runtime Atmosphere
- Device
- Accessibility

Semantic implementation may change.

Brand identity should not.

Users should instinctively recognise:

Mosaic

even if they cannot consciously explain why.

Brand And Atmosphere

One of the defining characteristics of Mosaic is the deliberate separation between:

Brand

and

Atmosphere.

Example.

Brand

↓

Stable Cyan Identity

Artwork.

Warm Orange Sunset

The Runtime Atmosphere should adopt:

- warm reflections
- warm acrylic
- warm lighting

while preserving:

- Brand identity
- Semantic hierarchy
- Accessibility

Atmosphere reflects entertainment.

Brand remains Mosaic.

Brand And Hero

The Hero should almost never use the Brand Colour directly.

Instead:

Artwork provides emotional identity.

Brand provides product identity.

The Hero therefore becomes emotionally expressive without becoming visually inconsistent.

This distinction prevents the interface from competing with the media.

Brand Across Products

Future Mosaic products should inherit the same Brand Architecture.

Examples include:

Mosaic Core

↓

Cyan

Mosaic Docs

↓

Different Supporting Palette

↓

Shared Brand Identity

Mosaic SDK

↓

Different Supporting Palette

↓

Shared Core Brand

Subsystems may evolve their own secondary palettes while remaining unmistakably part of the Mosaic ecosystem.

This approach creates a coherent family of products rather than isolated brands.

Brand Tokens

Future Semantic Tokens are expected to expose:

[text]

Brand.Primary

Brand.Secondary

Brand.Accent

Brand.Inverse

Applications should consume these Semantic Tokens.
They should never consume Primitive Brand Colours directly.

Runtime Rules

Runtime systems must never redefine Brand Tokens.

Examples.

Allowed.

Artwork

↓

Atmosphere

Not allowed.

Artwork

↓

Brand.Primary

Brand belongs to Mosaic.

Atmosphere belongs to entertainment.

Maintaining this separation is a fundamental architectural rule.

Accessibility

Brand Colours should always satisfy accessibility requirements.

Brand recognition should never depend upon:

- saturation
- contrast alone
- colour alone

Every branded interaction should remain understandable through:

- hierarchy
- typography
- composition
- iconography

Colour reinforces understanding.

It never replaces it.

Good Examples

Splash Screen

Neutral textured background.

Core Brand.

Subtle atmosphere.

No entertainment artwork.

The user immediately recognises Mosaic.

Runtime

Artwork dominates.

Brand appears only through:

- active controls
- subtle progress
- focused actions

Identity remains.

Entertainment leads.

Documentation

Brand becomes significantly more visible.

Entertainment artwork is absent.

Brand naturally assumes greater responsibility.

Anti-patterns

Brand Everywhere

Every surface adopts Brand Colours.

Artwork loses emotional impact.

Brand Mutation

Artwork changes the Brand Palette.

Users stop recognising Mosaic.

Marketing Interface

Large branded gradients dominate entertainment.

The interface begins competing with content.

Colour Branding

Brand identity depends entirely upon one saturated colour.

Identity should emerge from the complete Design System.

Not one pigment.

Conceptual Model

[mermaid]
flowchart TD

```
Brand
Brand --> Semantic
Semantic --> RuntimeAtmosphere["Runtime Atmosphere"]
RuntimeAtmosphere["Runtime Atmosphere"] --> Presentation
Presentation
Presentation --> Entertainment
```

Brand always enters before Runtime.

Runtime should never overwrite Brand.

Relationship To Future Specifications

Future specifications will formalise:

- exact palette values
- colour scales
- contrast ratios
- runtime blending
- acrylic interaction
- adaptive materials

This chapter defines only the architectural role of Brand Colours.

Summary

The Mosaic Brand should feel confident rather than loud.

Recognisable rather than repetitive.

Premium rather than promotional.

The strongest brand is not the one users notice constantly.

It is the one they recognise instantly.

Brand provides identity.

Artwork provides emotion.

The Colour System exists to ensure those two ideas remain beautifully separate.

Review Status

Status

Draft

Next File

03-semantic-colours.md

Semantic Colours

Purpose

Primitive colours represent physical values.

Brand colours represent Mosaic.

Semantic Colours represent **meaning**.

This distinction is one of the most important architectural separations within the Mosaic Colour System.

A component should never ask:

"Which colour should I use?"

Instead it should ask:

"What does this element mean?"

The Design System determines the correct colour automatically.

Definition

Within MDS, a **Semantic Colour** is defined as:

A colour role that communicates design meaning independently from any physical colour value.

Semantic Colours intentionally describe:

- purpose
- hierarchy
- interaction
- state

They intentionally avoid describing:

- cyan
- blue
- red
- grey

Implementation belongs to Primitive Colours.

Meaning belongs to Semantic Colours.

Why Semantic Colours Exist

Imagine two interfaces.

Poor.

```
[css]
background: #06B6D4;
color: #FFFFFF;
```

Questions immediately appear.

- Why cyan?
- Should every button be cyan?
- Does this colour communicate importance?
- Can it change?

Now compare.

```
[text]
Action.Primary
```

Every future implementation immediately understands the design intent.

Meaning survives.

Implementation evolves.

Semantic Before Colour

The Colour System intentionally follows the same philosophy as the Token Architecture.

Meaning

↓

Semantic Colour

↓

Primitive Colour

↓

Rendered Colour

Colour should always be the final consequence.

Never the starting point.

Semantic Categories

The Mosaic Colour System currently defines the following semantic groups.

```
[text]
Semantic
```

■■■ Surface

■■■ Text

■■■ Border

■■■ Icon

■■■ Action

■■■ Status

■■■ Focus

■■■ Selection

■■■ Playback

■■■ Navigation

■■■ Atmosphere

Every category answers a different question.

Surface

Purpose.

Communicate structural hierarchy.

Examples.

Surface.Canvas

Surface.Primary

Surface.Secondary

Surface.Hero

Surface.Overlay

Surface.Modal

Surface Colours should communicate:

- depth
- grouping
- emphasis

They should not communicate status.

Text

Purpose.

Communicate reading hierarchy.

Examples.

Text.Primary

Text.Secondary

Text.Tertiary

Text.Disabled

Text.Inverse

Text Colours should never depend upon decorative colour.

Hierarchy should remain understandable even if viewed in greyscale.

Border

Purpose.

Communicate separation.

Examples.

`Border.Subtle`

`Border.Strong`

`Border.Focus`

Borders should quietly reinforce structure.

They should rarely become primary visual elements.

Icon

Purpose.

Communicate symbolic hierarchy.

Examples.

`Icon.Primary`

`Icon.Secondary`

`Icon.Active`

`Icon.Disabled`

Icons should inherit semantic meaning from surrounding context wherever practical.

Action

Purpose.

Communicate interaction.

Examples.

`Action.Primary`

`Action.Secondary`

`Action.Destructive`

`Action.Passive`

Action Colours should communicate affordance.

Not decoration.

Status

Purpose.

Communicate state.

Examples.

Status.Success

Status.Warning

Status.Error

Status.Information

Status Colours should never become the sole mechanism communicating status.

Icons.

Typography.

Hierarchy.

Motion.

All should reinforce the same understanding.

Focus

Purpose.

Communicate current attention.

Examples.

Focus.Primary

Focus.Secondary

Focus.Background

Focus Colours reinforce the Composition Model.

They should never replace it.

Selection

Purpose.

Communicate user ownership.

Examples.

Selection.Active

Selection.Hover

Selection.Current

Selection Colours should feel calm.

They should never overwhelm the Hero.

Playback

Purpose.

Communicate media consumption.

Examples.

`Playback.Progress`

`Playback.Current`

`Playback.Completed`

Playback Colours should naturally integrate with the current Runtime Atmosphere.

They should not become independent branding elements.

Navigation

Purpose.

Communicate orientation.

Examples.

`Navigation.Active`

`Navigation.Inactive`

`Navigation.Current`

Navigation Colours should reinforce Anchors.

They should not compete with current Focus.

Semantic Colours Are Stable

Semantic Colour names should remain stable for many years.

Example.

`Surface.Hero`

may resolve to:

`Dark Theme`

↓

`Slate`

or

`Light Theme`

↓

`Paper White`

or

Runtime Atmosphere

↓

Artwork Acrylic

The semantic role remains identical.

Semantic Colours Are Layered

Semantic Colours inherit from Primitive Colours.

Example.

Surface.Hero

↓

Primitive.Surface.Dark

The consuming component should never know which Primitive Colour was selected.

Meaning remains separated from implementation.

Semantic Colours Never Reference Brand

Incorrect.

Brand.Cyan.Primary

Correct.

Action.Primary

Brand identity should emerge from the Brand System.

Not Semantic Colour names.

This separation is essential for future branding evolution.

Runtime Participation

Semantic Colours participate in Runtime Resolution.

Example.

Surface.Hero

↓

Runtime.Atmosphere

↓

Resolved Colour

The semantic meaning remains stable.

Only the implementation adapts.

Accessibility

Every Semantic Colour must remain accessible.

Accessibility should never require introducing new semantic concepts.

Instead.

`Text.Primary`

↓

Higher Contrast

The token remains identical.

Accessibility simply changes its implementation.

Good Examples

`Surface.Canvas`

↓

Neutral Background

`Action.Primary`

↓

Current Brand Accent

`Playback.Progress`

↓

Artwork-Aware Highlight

Each token communicates intent.

Implementation remains independent.

Anti-patterns

Colour Names

`Blue.Primary`

`Green.Success`

Meaning depends upon colour.

Component Colours

`Button.Blue`

Meaning depends upon implementation.

Brand Leakage

Netflix.Red

Semantic architecture has become product-specific.

Decorative Semantics

Pretty.Gradient

Semantic Tokens should communicate understanding.

Not aesthetics.

Semantic Model

[mermaid]
flowchart TD

```
Primitive
Primitive --> SemanticColours["Semantic Colours"]
SemanticColours["Semantic Colours"] --> Composition
Composition --> Runtime
Runtime --> Presentation
```

Semantic Colours provide meaning.

Everything above them communicates that meaning.

Everything below them implements it.

Relationship To Future Specifications

Future specifications define:

- Brand Palette
- Runtime Atmosphere
- Material System
- Component Library
- Theme Generation

All should consume Semantic Colours rather than introducing independent colour systems.

Summary

Semantic Colours represent one of the strongest architectural guarantees within the Mosaic Design System.

They ensure that:

- meaning survives redesign,
- accessibility survives theming,

- runtime adaptation preserves consistency,
- artwork enriches rather than replaces identity.

Every future implementation should therefore consume Semantic Colour roles rather than physical colour values.

That separation allows Mosaic to evolve for years without losing its visual language.

Review Status

Status

Draft

Next File

04-runtime-atmosphere.md

Runtime Atmosphere

Purpose

Runtime Atmosphere is one of the defining characteristics of the Mosaic Design System.

It is the mechanism through which the interface quietly reflects the emotional tone of the user's current entertainment without surrendering its own identity.

Unlike themes, which are selected by users...

or branding, which remains constant...

Runtime Atmosphere continuously adapts.

Its purpose is not to decorate the interface.

Its purpose is to increase immersion.

Definition

Within MDS, **Runtime Atmosphere** is defined as:

The adaptive environmental colour system generated from the user's current World and applied subtly throughout the interface.

Atmosphere exists to create emotional continuity between:

- artwork
- content
- interface

while preserving:

- accessibility
- semantic meaning
- brand identity

Why Runtime Atmosphere Exists

Entertainment already possesses atmosphere.

Examples include:

- album artwork
- anime key visuals
- film posters
- book covers
- promotional art

Traditional applications typically ignore this.

Every title appears inside the same interface.

Mosaic intentionally embraces the emotional identity already present in media.

Instead of recolouring the interface...

It reflects the media.

Reflection

Runtime Atmosphere should be thought of as **reflection** rather than **replacement**.

Incorrect.

Artwork

↓

Entire Interface

↓

Artwork Colours

Correct.

Artwork

↓

Atmosphere

↓

Subtle Reflection

↓

Interface

The interface should feel illuminated by the artwork.

Not painted with it.

This distinction is fundamental.

Atmosphere Is Environmental

Atmosphere belongs to the environment.

Not individual components.

Example.

Poor.

Button

↓

Artwork Colour

Preferred.

Environment

↓

Artwork Reflection

↓

Components Inherit Atmosphere

Components should remain semantically stable.

Atmosphere surrounds them.

Inputs

Runtime Atmosphere may evaluate:

[text]

Current Focus

↓

Current Artwork

↓

Dominant Palette

↓

Luminance

↓

Contrast

↓

Theme

↓

Accessibility

These inputs collectively determine the environmental atmosphere.

No single input possesses authority.

Outputs

Runtime Atmosphere produces conceptual outputs.

Examples include:

Atmosphere.Primary

Atmosphere.Secondary

Atmosphere.Highlight

Atmosphere.Glow

Future Material specifications determine how these outputs become visible.

Atmosphere Is Contextual

The same artwork may produce different Atmospheres depending upon Context.

Example.

Browsing.

Artwork



Subtle Reflection

Playback.

Artwork



Reduced Reflection

Reading.

Artwork



Soft Reflection

The artwork remains identical.

The user's activity changes.

Atmosphere should respect that activity.

Atmosphere Never Becomes Brand

Brand and Atmosphere intentionally remain separate.

Example.

Brand



Stable Identity

Atmosphere



Adaptive Emotion

Artwork should never redefine:

- Brand.Primary
- Brand.Secondary

- Brand.Accent

Those tokens belong exclusively to Mosaic.

Atmosphere enhances.

Brand identifies.

Atmosphere Should Be Peripheral

Users should rarely consciously notice Runtime Atmosphere.

Instead they should simply feel:

- warmer
- calmer
- brighter
- darker
- more immersive

Atmosphere should exist primarily within peripheral vision.

The entertainment remains the emotional centre.

Adaptive Intensity

Atmosphere intensity should vary according to Context.

Examples.

Playback

↓

Low Intensity

Browsing

↓

Medium Intensity

Hero

↓

Highest Intensity

Atmosphere should never reduce readability.

Understanding always possesses higher priority than immersion.

Atmosphere Regions

Future implementations may divide Atmosphere into conceptual regions.

Hero

↓

Primary Reflection

Canvas

↓

Supporting Reflection

Navigation

↓

Minimal Reflection

Overlay

↓

Neutral

This regional approach prevents atmosphere from overwhelming the interface.

Atmosphere And Materials

Atmosphere should normally appear through Materials.

Examples include:

- acrylic
- translucency
- glow
- subtle gradients
- reflected highlights

Flat colour replacement should generally be avoided.

Atmosphere should feel like light interacting with materials.

Not paint replacing them.

This directly supports the Refraction System defined elsewhere within Mosaic.

Atmosphere And Refraction

The Runtime Atmosphere becomes one of the primary inputs into the Mosaic Refraction System.

Conceptually.

[text]
Artwork

↓

Colour Extraction

↓

Runtime Atmosphere

↓

Material Refraction

↓

Rendered Surface

Notice that artwork never directly colours interface elements.

Instead it influences how light appears to travel through the interface.

This distinction gives Mosaic a recognisable visual identity that remains independent of any particular artwork.

Atmosphere Persistence

Atmosphere should evolve gradually.

Poor.

Artwork Changes

↓

Entire Interface Changes

Preferred.

Artwork Changes

↓

Atmosphere Blends

↓

Materials Adapt

↓

Composition Continues

Users should experience atmosphere as environmental change.

Not theme switching.

Accessibility

Runtime Atmosphere must never reduce accessibility.

Accessibility always overrides atmosphere.

If an atmosphere would reduce:

- contrast
- readability
- recognisability

its intensity should reduce automatically.

Accessibility is considered a higher-order design constraint.

Performance

Atmosphere should be computationally inexpensive.

Future implementations should:

- cache extracted palettes
- reuse generated atmosphere
- update only when Focus changes
- avoid recomputation during ordinary interaction

Atmosphere should never noticeably impact interaction performance.

Immersion should not come at the expense of responsiveness.

Good Examples

Hero Artwork

Artwork subtly illuminates surrounding acrylic surfaces.

Brand remains unchanged.

Typography remains readable.

Atmosphere feels natural.

Dark Science Fiction

Cool blue reflections appear within Hero materials.

The rest of the interface remains restrained.

The experience feels cohesive.

Warm Fantasy Novel

Soft amber reflections appear behind the Hero.

Supporting surfaces remain largely neutral.

Attention remains on the book cover.

Anti-patterns

Full Recolour

Every interface element adopts artwork colours.

Brand identity disappears.

Saturated UI

Atmosphere becomes more visually interesting than the entertainment itself.

The interface now competes with content.

Instant Theme Switching

Artwork changes produce abrupt colour changes.

Continuity is lost.

Accessibility Compromise

Atmosphere reduces readability.

Immersion should never weaken usability.

Atmosphere Model

[mermaid]
flowchart TD

```
Artwork
Artwork --> PaletteExtraction["Palette Extraction"]
PaletteExtraction["Palette Extraction"] --> RuntimeAtmosphere["Runtime Atmosphere"]
RuntimeAtmosphere["Runtime Atmosphere"] --> MaterialSystem["Material System"]
MaterialSystem["Material System"] --> Presentation
Presentation --> Immersion
```

Atmosphere exists between artwork and materials.

It does not bypass the Material System.

Relationship To Future Specifications

Future specifications will formalise:

- palette extraction
- UV refraction
- acrylic rendering
- glow synthesis
- adaptive blending
- material shaders

MDS-002 defines only the conceptual role of Runtime Atmosphere.

Future MDS specifications define how it is implemented.

Summary

Runtime Atmosphere is one of the defining innovations of the Mosaic Design System.

Rather than colouring the interface...

It allows the user's entertainment to illuminate it.

Brand remains stable.

Meaning remains stable.

Accessibility remains stable.

Only the atmosphere evolves.

When successful, users should feel that the interface belongs to their current entertainment without ever feeling that it has stopped being unmistakably Mosaic.

Review Status

Status

Draft

Next File

05-artwork-colour-extraction.md

Artwork Colour Extraction

Purpose

Runtime Atmosphere depends upon understanding the visual identity of the user's current entertainment.

This chapter defines how artwork should contribute to the Colour System.

Importantly...

Artwork should never directly determine interface colours.

Instead, artwork provides **environmental influence**.

This distinction allows Mosaic to remain recognisable while still feeling deeply connected to the media currently occupying the user's World.

Philosophy

The objective of Artwork Colour Extraction is **not colour matching**.

The objective is **atmosphere generation**.

The Design System should never ask:

"What colour is this poster?"

Instead it should ask:

"What kind of light would this artwork cast into the room?"

This philosophy directly supports the Refraction System established elsewhere within the Mosaic Design Language.

Definition

Within MDS, **Artwork Colour Extraction** is defined as:

The process of deriving stable atmospheric characteristics from entertainment artwork for use by the Runtime Atmosphere system.

Artwork Extraction produces:

- atmosphere
- reflection
- lighting influence

It does **not** produce:

- themes
- brand colours
- semantic colours

The Separation Principle

Artwork influences.

It does not control.

The Colour System therefore follows this pipeline.

[text]
Artwork

↓

Visual Analysis

↓

Atmosphere Model

↓

Runtime Atmosphere

↓

Materials

↓

Presentation

Notice that Semantic Colours are intentionally absent.

Artwork never bypasses semantic meaning.

Raw Colour Is Not Enough

Simply extracting dominant colours produces poor results.

Example.

A movie poster containing:

- bright red logo
- tiny blue sky
- black clothing

might incorrectly produce:

Bright Red Interface

This is undesirable.

Artwork extraction should instead understand:

- colour balance
- visual weight
- luminance

- saturation
- spatial distribution

The objective is atmosphere.

Not averaging pixels.

Colour Characteristics

Future implementations should derive several conceptual properties.

Dominant Hue

The overall emotional direction of the artwork.

Accent Hue

Strong supporting colours.

Used sparingly.

Luminance

Overall brightness.

Important for:

- atmosphere
- acrylic behaviour
- readability

Saturation

Overall colour intensity.

High saturation should generally be reduced before entering Runtime Atmosphere.

Entertainment artwork often contains stronger colours than suitable interface materials.

Temperature

Whether the artwork feels:

- warm
- cool
- neutral

Temperature strongly influences perceived atmosphere.

Contrast Profile

Describes the visual energy of the artwork.

High contrast artwork should not automatically produce high contrast interfaces.

Instead, contrast influences how strongly atmosphere is expressed.

Palette Generation

The extraction system should produce a constrained palette.

Conceptually.

[text]
Artwork

↓

Analysis

↓

Primary Atmosphere

Secondary Atmosphere

Accent Reflection

Neutral Balance

Future specifications define the exact algorithms.

MDS defines only the conceptual outputs.

Reflection Rather Than Sampling

Artwork should be treated as a light source.

Not a paint bucket.

Poor.

Artwork

↓

Sample Colour

↓

Interface

Preferred.

Artwork

↓

Ambient Light

↓

Reflection

↓

Materials

This distinction gives Mosaic its distinctive visual identity.

The interface feels illuminated by the media rather than recoloured by it.

Stability

Artwork analysis should remain stable.

Minor artwork changes should not produce dramatically different atmospheres.

Future implementations should therefore favour:

- smoothing
- averaging
- constrained adaptation

over:

- immediate pixel sampling

Atmosphere should feel calm.

Not reactive.

Hero Artwork

The Hero Artwork receives the greatest influence.

Examples.

Current Film

↓

Hero Atmosphere

Current Book

↓

Hero Reflection

Current Album

↓

Hero Glow

Peripheral artwork should contribute significantly less.

The current Focus always possesses the strongest environmental influence.

Multiple Artwork Sources

Future Compositions may contain many artworks simultaneously.

Examples include:

- Continue Watching
- Related Works
- Collections
- Recommendations

Only one artwork should normally drive Runtime Atmosphere.

The Hero.

Supporting artwork should remain visually independent.

Allowing multiple artworks to influence atmosphere simultaneously would weaken coherence.

Artwork Changes

Changing Focus naturally changes artwork.

Atmosphere should therefore evolve.

Preferred.

[text]
Old Atmosphere

↓

Blend

↓

New Atmosphere

Avoid.

[text]
Old Atmosphere

↓

Instant Replacement

↓

New Atmosphere

Blending preserves continuity.

Accessibility

Artwork extraction must never reduce accessibility.

Examples.

Highly saturated artwork.

↓

Reduced saturation.

Very dark artwork.



Maintain minimum contrast.

Very bright artwork.



Protect readability.

Artwork always remains subordinate to accessibility.

Performance

Artwork analysis should occur infrequently.

Recommended conceptual triggers include:

- Hero changes
- Focus changes
- artwork replacement

Ordinary scrolling or interaction should not repeatedly analyse artwork.

Future runtime systems should cache extracted atmosphere aggressively.

Plugins

Extensions may contribute artwork.

They should never contribute atmosphere.

Example.

Anime Plugin.

Poster



Artwork

Platform.

Artwork



Atmosphere

This preserves one coherent visual language regardless of content source.

Good Examples

Science Fiction

Artwork.

Cool blue lighting.

Result.

Cool reflections.

Neutral surfaces.

Stable branding.

Fantasy

Artwork.

Warm amber lighting.

Result.

Subtle amber acrylic.

Hero emphasis.

Brand unchanged.

Music

Album artwork.

Rich magenta accents.

Result.

Very subtle magenta environmental glow.

Controls remain semantically coloured.

Anti-patterns

Dominant Colour Sampling

Largest colour immediately recolours the interface.

Atmosphere becomes unstable.

Multi-Artwork Mixing

Several posters simultaneously influence the interface.

The atmosphere becomes visually noisy.

Brand Replacement

Artwork changes Brand colours.

Users stop recognising Mosaic.

Unbounded Saturation

Artwork saturation passes directly into the interface.

Readability suffers.

The interface competes with the media.

Extraction Model

[mermaid]
flowchart TD

```
HeroArtwork
HeroArtwork --> ArtworkAnalysis
ArtworkAnalysis --> AtmosphereModel
AtmosphereModel --> RuntimeAtmosphere
RuntimeAtmosphere --> MaterialSystem
MaterialSystem --> Presentation
```

Artwork influences the environment.

It never directly colours components.

Relationship To Future Specifications

Future specifications will define:

- dominant colour extraction
- palette clustering
- luminance balancing
- temporal blending
- UV refraction integration
- GPU acceleration
- cache invalidation

These implementation details intentionally remain outside the scope of MDS-002.

Summary

Artwork Colour Extraction exists to transform entertainment into atmosphere.

Not into interface.

The platform should feel as though light from the user's entertainment is subtly interacting with the Mosaic materials.

This creates emotional continuity while preserving:

- brand identity
- accessibility
- semantic meaning
- compositional clarity

That balance is one of the defining characteristics of the Mosaic visual language.

Review Status

Status

Draft

Next File

06-theme-architecture.md

Theme Architecture

Purpose

The Mosaic Colour System intentionally separates:

- Brand
- Semantic Meaning
- Runtime Atmosphere

The Theme Architecture defines how these independent systems combine into a coherent visual experience.

Unlike many applications, a Theme within Mosaic is **not** a palette.

A Theme is a structured interpretation of the Design System.

Themes should therefore alter implementation while preserving meaning.

Definition

Within MDS, a **Theme** is defined as:

A deterministic mapping between semantic design intent and a specific visual implementation.

A Theme does not define:

- hierarchy
- behaviour
- composition
- interaction

Those concepts already exist.

A Theme simply determines how they appear.

Why Themes Exist

Users expect software to adapt to different environments.

Examples include:

- Light Mode
- Dark Mode
- High Contrast
- OLED Displays
- Accessibility

Rather than maintaining separate design systems for each environment, Mosaic defines one semantic architecture capable of producing many themes.

The user's World remains constant.

The Theme changes.

Themes Are Interpretations

Every Theme is an interpretation of the same semantic language.

Example.

[text]
Surface.Canvas

↓

Light Theme

↓

Warm Paper

[text]
Surface.Canvas

↓

Dark Theme

↓

Deep Slate

[text]
Surface.Canvas

↓

OLED Theme

↓

Pure Black

The semantic meaning remains unchanged.

Only the visual interpretation differs.

Theme Hierarchy

Every Theme is constructed from the same conceptual layers.

[text]
Brand

↓

Semantic Colours

↓

Runtime Atmosphere

↓

Accessibility

↓

Resolved Theme

Each layer contributes one responsibility.

No layer should replace another.

Theme Responsibilities

Themes are responsible for:

- colour mapping
- contrast
- luminance
- neutral surfaces
- runtime blending
- accessibility adaptation

Themes are **not** responsible for:

- hierarchy
- layout
- component behaviour
- interaction

These responsibilities belong elsewhere within MDL and MDS.

Base Themes

The Colour System currently defines two foundational themes.

Light

and

Dark

Every future theme should inherit from one of these foundations.

This ensures long-term consistency while allowing implementation to evolve.

Derived Themes

Future implementations may introduce derived themes.

Examples include:

Dark

↓

OLED

Dark

↓

Cinema

Light

↓

Paper

These remain visual interpretations rather than independent design systems.

Runtime Layering

Runtime Atmosphere sits above the Theme.

Conceptually.

[text]

Theme

↓

Runtime Atmosphere

↓

Presentation

This ordering is intentional.

Changing artwork should never require regenerating the Theme.

Instead the Theme provides stable foundations upon which Runtime Atmosphere operates.

Neutral Foundations

Every Theme should begin with a neutral foundation.

Neutral surfaces provide:

- readability
- accessibility
- restraint
- flexibility

Atmosphere then adds subtle environmental influence.

The interface should therefore feel illuminated rather than recoloured.

Brand Consistency

Brand Colours should remain recognisable across every Theme.

Examples.

Brand.Primary

↓

Light Theme

↓

Adjusted Luminance

Brand.Primary

↓

Dark Theme

↓

Adjusted Contrast

The Brand adapts for accessibility.

It does not change identity.

Theme Inheritance

Themes inherit from Semantic Tokens.

They do not redefine them.

[text]
Semantic

↓

Theme

↓

Resolved Colour

Applications therefore consume identical Semantic Tokens regardless of the active Theme.

Accessibility

Accessibility modifies Themes rather than replacing them.

Example.

Dark Theme

↓

High Contrast

↓

Resolved Theme

This layered approach preserves:

- semantic meaning
- runtime atmosphere
- brand identity

while improving readability.

Theme Resolution

Future runtime systems should resolve themes using the following conceptual order.

[text]

Base Theme

↓

Accessibility

↓

Runtime Atmosphere

↓

Device

↓

Presentation

This ordering ensures accessibility always possesses higher authority than atmosphere.

Theme Persistence

Theme changes should preserve continuity.

Poor.

Dark

↓

Light

↓

Flash

↓

Complete Repaint

Preferred.

Dark

↓

Blend

↓

Light



Atmosphere Re-evaluates

Theme transitions should feel calm.

Not dramatic.

Themes Across Devices

Every client should communicate the same Theme.

Desktop.



Desktop implementation.

Television.



Television implementation.

Mobile.



Mobile implementation.

The visual language remains recognisably Mosaic.

Only presentation adapts.

Plugins

Extensions should never provide themes.

They consume them.

Plugins inherit:

- Semantic Colours
- Runtime Atmosphere
- Accessibility
- Brand

This ensures every extension automatically participates in future redesigns.

Good Examples

Light Theme

Neutral canvas.

Subtle shadows.

Restrained atmosphere.

Brand accents.

Dark Theme

Deep neutral surfaces.

Artwork-driven reflections.

High readability.

Stable branding.

OLED Theme

Pure black foundation.

Reduced luminance.

Atmosphere preserved.

Hierarchy unchanged.

Anti-patterns

Theme-Specific Semantics

`Dark.Surface.Primary`

Semantic meaning has leaked into themes.

Runtime Themes

Artwork generating entirely new themes.

Atmosphere should refine.

Not replace.

Brand Themes

Every theme changes brand identity.

Recognition decreases.

Plugin Themes

Extensions introducing independent visual identities.

The ecosystem fragments.

Theme Model

[mermaid]
flowchart TD

```
SemanticColours
SemanticColours --> BaseTheme
BaseTheme --> Accessibility
Accessibility --> RuntimeAtmosphere
RuntimeAtmosphere --> ResolvedTheme
ResolvedTheme --> Presentation
```

Themes translate semantic meaning into consistent visual language.

They do not redefine that meaning.

Relationship To Future Specifications

Future specifications will formalise:

- Light Theme
- Dark Theme
- OLED Theme
- Accessibility Themes
- Theme blending
- Runtime transitions
- Platform-specific generation

MDS-002 establishes only the architectural framework.

Summary

Themes are not collections of colours.

They are interpretations of the Mosaic Design Language.

Every Theme should communicate the same:

- hierarchy
- meaning
- identity

while allowing implementation to adapt to:

- environment

- accessibility
- user preference
- runtime atmosphere

The user's World remains constant.

The Theme simply provides the most appropriate visual expression of it.

Review Status

Status

Draft

Next File

07-light-and-dark.md

Light and Dark

Purpose

Light Mode and Dark Mode are often treated as independent themes.

Within Mosaic they are not.

They are two visual interpretations of the same semantic language.

This chapter defines how the Colour System should express identical understanding across both environments while preserving:

- hierarchy
- atmosphere
- accessibility
- brand identity

The objective is not visual parity.

The objective is conceptual parity.

Philosophy

Light Mode and Dark Mode should answer exactly the same questions.

Users should never need to relearn:

- hierarchy
- emphasis
- navigation
- interaction

because they changed theme.

Only the visual expression should change.

Understanding should remain identical.

One Design System

Mosaic intentionally rejects maintaining two independent colour systems.

Instead:

[text]
Semantic Colours

↓

Light Theme

or

Dark Theme

↓

Presentation

Both themes inherit identical semantic meaning.

Neither introduces additional concepts.

Light Mode

Light Mode is intended to feel:

- open
- calm
- breathable
- paper-like
- premium

Light Mode should avoid the stark, clinical appearance common in many productivity applications.

Instead it should feel like a softly illuminated gallery.

Neutral surfaces should dominate.

Artwork provides emotional colour.

Dark Mode

Dark Mode is intended to feel:

- immersive
- cinematic
- focused
- atmospheric
- comfortable

Dark Mode is expected to become the primary mode for media consumption.

However...

It should not become:

- overly saturated
- excessively high contrast
- visually aggressive

The interface should quietly recede behind the media.

Equal Hierarchy

The following concepts should possess identical hierarchy across both themes.

Hero

↓

Supporting

↓

Contextual

↓

Peripheral

Changing themes should never alter compositional understanding.

Only implementation changes.

Surface Relationships

Surface relationships should remain consistent.

Example.

Canvas

↓

Primary

↓

Secondary

↓

Overlay

↓

Hero

The visual implementation changes.

The relationships remain identical.

Users should recognise the same structural language regardless of theme.

Atmosphere Behaviour

Runtime Atmosphere should adapt differently depending upon the active theme.

Light Mode

Atmosphere should appear primarily through:

- subtle colour reflections

- warm paper illumination
- restrained acrylic
- soft gradients

Dark Mode

Atmosphere should appear primarily through:

- reflected light
- luminous acrylic
- ambient glow
- subtle edge illumination

The emotional effect remains similar.

The implementation differs.

Contrast

Contrast should support readability.

Not dominate the experience.

Light Mode.

Contrast should feel:

- gentle
- comfortable
- editorial

Dark Mode.

Contrast should feel:

- cinematic
- focused
- restrained

Both should satisfy accessibility requirements.

Neither should rely upon extreme black-and-white contrast for ordinary interface elements.

Neutral Foundations

Both themes should begin with neutral foundations.

Light.

Soft Paper

Dark.

Deep Slate

These foundations provide space for:

- artwork
- runtime atmosphere
- semantic accents

The interface remains visually calm regardless of theme.

Brand Behaviour

Brand identity should remain stable.

Example.

`Brand.Primary`

Light.



Adjusted luminance.

Dark.



Adjusted contrast.

The colour may change physically.

The brand should remain immediately recognisable.

Hero Behaviour

The Hero should remain the emotional centre in both themes.

Light Mode.

Artwork appears naturally illuminated.

Dark Mode.

Artwork appears naturally surrounded by reflected light.

Neither implementation should overpower the artwork itself.

Acrylic Behaviour

Future Material specifications are expected to implement Acrylic differently.

Light Mode.

- lighter translucency

- softer blur
- brighter refraction

Dark Mode.

- deeper translucency
- stronger reflected highlights
- richer atmospheric depth

The material changes.

The compositional role remains identical.

Accessibility

Accessibility possesses higher authority than theme.

If a conflict exists between:

- runtime atmosphere
- theme
- readability

Accessibility should always win.

Light and Dark Modes should therefore expose identical semantic information regardless of visual implementation.

Theme Switching

Switching between Light and Dark should feel like environmental adaptation.

Not application replacement.

Preferred.

[text]
Dark

↓

Blend

↓

Light

↓

Atmosphere Re-evaluates

↓

Materials Update

Avoid.

[text]

Dark

↓

Flash

↓

Entire Interface Repaints

Continuity should remain uninterrupted.

Automatic Themes

Future implementations may support automatic theme selection.

Examples include:

- system preference
- sunset
- ambient light
- bedtime routines

These behaviours should influence only the active theme.

They should never affect:

- hierarchy
- composition
- interaction
- semantic meaning

Cross-Device Behaviour

Light and Dark Modes should remain conceptually identical across:

- desktop
- mobile
- television
- tablet

Different display technologies may require different implementation values.

The design language should remain recognisable.

Plugins

Extensions should remain completely unaware of Light and Dark Modes.

Plugins consume:

- Semantic Colours
- Runtime Tokens

The Theme Resolver determines final implementation.

This guarantees that every extension automatically supports:

- Light Mode
- Dark Mode
- future themes

without additional development.

Good Examples

Reading

Light Mode.

Soft paper surfaces.

Gentle reflected artwork.

Minimal interface.

Playback

Dark Mode.

Deep neutral canvas.

Subtle artwork reflections.

Controls quietly illuminated.

Administration

Light Mode.

Clear hierarchy.

Minimal atmosphere.

Strong readability.

Entertainment and administration remain visually related without becoming identical.

Anti-patterns

Separate Design Systems

Light and Dark Modes introducing different hierarchy.

Colour Inversion

Simply inverting colours.

The emotional character of the interface is lost.

Saturated Dark Mode

Dark surfaces overwhelmed by artwork colours.

The interface becomes visually noisy.

Pure White

Large regions of absolute white.

The interface feels clinical.

Pure Black

Large regions of absolute black outside OLED-specific themes.

Visual depth decreases.

Atmosphere becomes difficult to communicate.

Theme Behaviour Model

[mermaid]
flowchart TD

```
SemanticColours  
SemanticColours --> LightOrDark  
LightOrDark --> RuntimeAtmosphere  
RuntimeAtmosphere --> MaterialSystem  
MaterialSystem --> Presentation
```

Theme determines environmental interpretation.

Atmosphere enriches that interpretation.

Neither changes semantic meaning.

Relationship To Future Specifications

Future specifications will define:

- exact neutral scales
- luminance curves
- acrylic differences
- OLED behaviour
- HDR displays

- colour blending

This chapter establishes only the conceptual relationship between Light and Dark Modes.

Summary

Light Mode and Dark Mode are two expressions of one design language.

Users should immediately recognise:

- the same hierarchy
- the same interaction
- the same composition
- the same companion

regardless of theme.

Only the environment changes.

The user's World remains exactly the same.

Review Status

Status

Draft

Next File

08-accessibility.md

Accessibility

Purpose

Accessibility is not an optional enhancement to the Mosaic Colour System.

It is a primary design constraint.

Every colour decision within Mosaic should remain understandable regardless of a user's:

- visual ability
- environmental conditions
- display technology
- colour perception
- accessibility preferences

Accessibility should therefore be considered an architectural property rather than a post-processing step.

Philosophy

The Colour System follows one fundamental rule.

Understanding must never depend upon colour alone.

Colour may reinforce understanding.

It must never become the only mechanism communicating:

- hierarchy
- interaction
- status
- progress
- warnings
- selection

Removing colour should reduce visual richness.

It should not destroy comprehension.

Accessibility Before Atmosphere

One of the defining priorities of the Mosaic Colour System is:

[text]

Accessibility

↓

Semantic Meaning

↓

Brand

↓

Atmosphere

If Runtime Atmosphere reduces accessibility...

Atmosphere should adapt.

Not the other way around.

Accessibility is considered a higher-order architectural constraint.

Semantic Stability

Accessibility should never introduce new semantic concepts.

Example.

Correct.

```
[text]  
Text.Primary
```

↓

Higher Contrast

Incorrect.

```
[text]  
Accessibility.Text.Primary
```

Semantic meaning remains identical.

Only implementation changes.

Contrast

Contrast exists to communicate hierarchy.

Not visual drama.

Every semantic colour role should satisfy minimum readability requirements under:

- Light Mode
- Dark Mode
- Runtime Atmosphere
- High Contrast

Future implementations should validate contrast automatically during build and runtime where appropriate.

Colour Independence

Every important interaction should remain understandable without relying upon colour.

Examples include:

Status

Instead of:

Green



Success

Use:

- icon
- label
- hierarchy
- colour

Together.

Selection

Instead of:

Blue Border

Use:

- position
- emphasis
- typography
- colour

Together.

Focus

Instead of:

Highlighted Colour

Use:

- composition
- spacing
- movement
- colour

Together.

Runtime Adaptation

Runtime Atmosphere should participate in accessibility.

Example.

Current artwork.



Low contrast.



Runtime reduces atmosphere intensity.



Semantic hierarchy remains readable.

Users should never need to choose between immersion and readability.

Colour Vision

The Colour System should avoid depending upon problematic colour combinations.

Examples include:

- red vs green
- blue vs purple
- yellow vs white

Semantic differentiation should always be reinforced through additional visual mechanisms.

Future validation tooling should automatically identify problematic combinations.

High Contrast

High Contrast Mode should be treated as another Theme.

It should inherit:

- semantic meaning
- hierarchy
- composition

while increasing:

- contrast
- edge definition
- readability

High Contrast should never become a separate Design System.

Motion And Colour

Motion should reinforce colour rather than compensate for it.

Example.

Status

↓

Colour

+

Icon

+

Subtle Motion

No single mechanism should communicate understanding independently.

Together they produce a significantly more accessible experience.

Typography

Colour should never compensate for weak typography.

Good typography should remain understandable before colour is applied.

Colour strengthens typography.

It does not rescue it.

Artwork

Artwork frequently contains poor accessibility characteristics.

Examples include:

- extremely low contrast
- excessive saturation
- colour imbalance

Runtime Atmosphere should automatically constrain artwork influence.

Artwork should enrich.

Never compromise.

OLED Displays

Future OLED-specific themes should preserve accessibility.

Pure black backgrounds should be introduced only where they improve readability or power efficiency.

Pure black should never become the default aesthetic if it weakens depth perception or visual hierarchy.

Reduced Colour

Future accessibility modes may intentionally reduce colour.

Example.

[text]
Runtime Atmosphere

↓

Minimal

The Composition should remain immediately understandable.

This serves as a useful validation technique for ensuring hierarchy does not depend upon colour.

Plugin Behaviour

Extensions inherit accessibility automatically.

Plugins should never implement independent:

- colour systems
- contrast systems
- themes

Instead they consume Semantic Tokens.

The platform guarantees accessible presentation.

Validation

Future tooling should validate:

- contrast ratios
- semantic consistency
- runtime adaptation
- theme compatibility
- colour vision deficiencies
- token usage

Accessibility should become part of continuous integration rather than manual review.

Good Examples

Hero

Artwork influences atmosphere.

Runtime reduces saturation.

Typography remains readable.

Brand remains recognisable.

Status

Success communicated through:

- icon
- label
- hierarchy
- colour

Removing colour does not remove understanding.

Playback

Progress communicated through:

- position
- motion
- composition
- semantic colour

Understanding remains intact across all themes.

Anti-patterns

Colour Only

Status communicated exclusively through colour.

Atmosphere First

Artwork reduces readability.

Decorative Contrast

Very high contrast used purely for dramatic effect.

Independent Accessibility

Plugins inventing their own accessibility behaviour.

The platform loses consistency.

Accessibility Model

[mermaid]
flowchart TD

```
SemanticMeaning
SemanticMeaning --> Accessibility
Accessibility --> Theme
Theme --> RuntimeAtmosphere
RuntimeAtmosphere --> Presentation
```

Accessibility always evaluates before atmospheric adaptation.

Understanding remains the highest priority.

Relationship To Future Specifications

Future specifications are expected to formalise:

- WCAG compliance strategy
- HDR adaptation
- colour vision simulation
- automated validation
- accessibility testing
- adaptive typography
- material accessibility

These systems should all build upon the principles established in this chapter.

Summary

Accessibility is not a constraint placed upon the Colour System.

It is one of its primary design objectives.

Every user should experience:

- the same hierarchy
- the same understanding
- the same companion
- the same World

regardless of:

- colour perception
- display technology
- accessibility preferences

The Colour System succeeds when colour becomes a powerful enhancement to understanding rather than a requirement for it.

Review Status

Status

Draft

Next File

09-colour-resolution.md

Colour Resolution

Purpose

Previous chapters established:

- Brand Colours
- Semantic Colours
- Runtime Atmosphere
- Theme Architecture
- Accessibility

This chapter defines how those independent systems become a single resolved colour presented to the user.

Colour Resolution is responsible for translating design intent into implementation while preserving every architectural guarantee established by the Mosaic Design Language.

Components should never determine colours themselves.

They should consume resolved semantic intent.

Definition

Within MDS, **Colour Resolution** is defined as:

The deterministic process through which semantic colour intent becomes an accessible, context-aware, platform-specific colour value.

Colour Resolution never creates meaning.

It only implements meaning.

Why Resolution Exists

Without Colour Resolution every component becomes responsible for answering questions such as:

- Which theme is active?
- Which artwork is visible?
- Is accessibility enabled?
- Which device is being used?
- Which atmosphere should apply?

This creates duplication.

Instead:

Component

↓

Semantic Colour

↓

Resolved Colour

All complexity remains inside the Colour System.

Resolution Pipeline

Every colour should follow the same conceptual resolution pipeline.

[text]

Primitive Palette

↓

Semantic Colour

↓

Theme

↓

Accessibility

↓

Runtime Atmosphere

↓

Platform Adaptation

↓

Resolved Colour

Each layer contributes one responsibility.

No layer duplicates another.

Resolution Order

Colour Resolution should always occur in the same order.

[text]

1.

Primitive Colour

↓

2.

Semantic Colour

↓

3.

Theme

↓

4.

Accessibility

↓

5.

Runtime Atmosphere

↓

6.

Device Adaptation

↓

7.

Resolved Output

Changing this order weakens architectural consistency.

Accessibility First

Accessibility should always possess higher authority than atmosphere.

Incorrect.

Artwork

↓

Low Contrast

↓

Unreadable Text

Correct.

Artwork

↓

Accessibility Validation

↓

Adjusted Atmosphere

↓

Readable Interface

Immersion should never reduce comprehension.

Semantic Stability

One of the most important guarantees of the Colour System is:

Semantic Meaning

↓

Never Changes

Example.

Surface.Hero

may resolve differently depending upon:

- theme
- artwork
- accessibility
- device

The semantic identity remains identical.

Applications should therefore consume only Semantic Colours.

Runtime Refinement

Runtime Atmosphere refines implementation.

It never replaces semantic meaning.

Example.

Surface.Hero

↓

Dark Theme

↓

Artwork Reflection

↓

Resolved Hero Surface

Atmosphere subtly influences the final result.

It never determines it completely.

Device Adaptation

Different displays possess different characteristics.

Examples include:

- OLED
- LCD
- HDR
- SDR
- eInk
- Projection

Device adaptation should compensate for hardware differences while preserving perceived meaning.

Users should experience one Colour System.

Not device-specific colour languages.

Resolution Is Deterministic

Given identical inputs...

Colour Resolution should always produce identical outputs.

Example.

Dark Theme

↓

Current Artwork

↓

TV

↓

Reduced Motion Disabled

↓

Resolved Colour

Every Mosaic client should produce visually equivalent results.

Determinism improves:

- testing
- caching
- accessibility
- predictability

Resolution Inputs

Conceptually, Colour Resolution evaluates:

[text]

Semantic Colour

↓

Theme

↓

Accessibility

↓

Atmosphere

↓

Display Characteristics

↓

User Preferences

These inputs refine implementation.

They do not alter semantic intent.

Fallback Behaviour

Every Semantic Colour should possess a valid fallback.

Example.

Artwork Missing

↓

Brand Neutrals

↓

Resolved Surface

Components should never receive unresolved colours.

The system should always produce a meaningful visual result.

Caching

Resolved colours should remain cacheable.

Example.

Artwork

↓

Atmosphere

↓

Resolved Palette

↓

Cache

The cache should invalidate only when:

- Focus changes
- artwork changes
- theme changes
- accessibility changes

Ordinary interaction should rarely require recolour resolution.

Lazy Resolution

Colour Resolution should occur only when colours are required.

Unused semantic colours should remain unresolved.

This improves runtime performance while preserving identical architectural behaviour.

Components

Components should consume only resolved Semantic Colours.

Correct.

Hero Tile

↓

Surface.Hero

Incorrect.

Hero Tile

↓

Determine Theme

↓

Determine Atmosphere

↓

Resolve Colour

Resolution belongs exclusively to the Colour System.

Extensions

Extensions should never resolve colours.

Plugins consume:

- Semantic Colours
- Runtime Tokens

The platform determines final colour values.

This guarantees future compatibility across:

- themes
- accessibility
- runtime atmosphere

without plugin modification.

Good Examples

Surface.Canvas

↓

Dark Theme

↓

Resolved Neutral Surface

Action.Primary

↓

Brand Accent

↓

Accessibility

↓

Resolved Action Colour

Surface.Hero

↓

Artwork Reflection

↓

Adaptive Acrylic

Each example preserves semantic intent.

Only implementation changes.

Anti-patterns

Component Colour Logic

Every component resolving colours independently.

Artwork Overrides

Artwork replacing Semantic Colours entirely.

Platform Resolution

Flutter, CSS or SwiftUI redefining semantic meaning.

Runtime Mutation

Runtime changing the identity of Semantic Colours.

Runtime refines implementation.

It never changes meaning.

Colour Resolution Model

```
[mermaid]
flowchart TD
    Primitive --> Semantic
    Semantic --> Theme
    Theme --> Accessibility
    Accessibility --> RuntimeAtmosphere["Runtime Atmosphere"]
    RuntimeAtmosphere["Runtime Atmosphere"] --> Platform
    Platform --> Presentation
```

Meaning flows downward.

Implementation never flows upward.

Relationship To Future Specifications

Future specifications are expected to define:

- palette blending
- atmosphere interpolation
- HDR colour mapping
- GPU colour pipelines
- platform renderers
- colour cache invalidation

This chapter defines the architectural process.

Future specifications define implementation.

Summary

Colour Resolution is the mechanism through which the Mosaic Colour System becomes visible.

Its responsibility is to preserve:

- semantic meaning
- accessibility
- runtime atmosphere
- brand identity

while producing one coherent visual language across every supported platform.

Components should simply ask:

"What does this colour mean?"

The Colour System answers everything else.

Review Status

Status

Draft

Next File

10-runtime-synthesis.md

Runtime Synthesis

Purpose

Previous chapters defined how colour should be:

- interpreted,
- organised,
- resolved.

This chapter defines the conceptual process through which those independent systems become one coherent runtime experience.

Unlike traditional theming systems, Mosaic does not simply select colours.

It synthesises an atmosphere.

Runtime Synthesis is the architectural process that combines:

- Brand
- Semantic Colour
- Runtime Atmosphere
- Theme
- Accessibility

into one visually coherent environment.

Definition

Within MDS, **Runtime Synthesis** is defined as:

The continuous generation of a coherent visual atmosphere by combining stable semantic design intent with dynamic runtime inputs.

Runtime Synthesis is not theme switching.

It is environmental adaptation.

Why Synthesis Exists

Traditional design systems generally behave like this.

[text]
Theme

↓

Static Colours

↓

Interface

Mosaic intentionally behaves differently.

[text]
Semantic Meaning

↓

Runtime Analysis

↓

Atmosphere

↓

Material Response

↓

Interface

The result is an interface that feels alive without becoming unpredictable.

Synthesis Is Layered

Runtime Synthesis intentionally occurs after semantic resolution.

[text]
Semantic Meaning

↓

Theme

↓

Accessibility

↓

Artwork Analysis

↓

Atmosphere

↓

Material Response

↓

Presentation

Meaning is established first.

Emotion is introduced afterwards.

Synthesis Inputs

Future runtime systems may combine information from multiple sources.

Examples include:

Current Artwork

↓

Current Focus

↓

Current Context

↓

Time Of Day

↓

Device

↓

Accessibility

↓

Theme

↓

Brand

Each contributes part of the final atmosphere.

No individual input should dominate the result.

Brand Stability

Brand identity should remain the most stable input.

Brand provides:

- recognition
- trust
- familiarity

Runtime should never alter:

- Brand.Primary
- Brand.Secondary
- Brand.Accent

Instead it synthesises atmosphere around them.

Brand is the architecture.

Atmosphere is the lighting.

Semantic Stability

Semantic colour meaning should also remain fixed.

Example.

Action.Primary

Runtime may alter:

- luminance
- saturation
- reflection

It must never alter:

- semantic responsibility
- interaction meaning
- hierarchy

Meaning remains stable.

Expression evolves.

Atmosphere Synthesis

Artwork contributes emotional information.

Not direct colour.

Example.

Hero Artwork

↓

Palette Analysis

↓

Atmosphere

↓

Reflection

↓

Materials

Notice that components never receive artwork colours directly.

Materials interpret atmosphere.

Material Response

Runtime Synthesis intentionally stops at Materials.

Example.

[text]
Atmosphere

↓

Acrylic

↓

Refraction

↓

Surface

The Material System decides:

- diffusion
- translucency
- glow
- reflection

The Colour System provides only environmental influence.

This separation allows future Material Systems to evolve independently.

Temporal Synthesis

Atmosphere should evolve gradually.

Preferred.

[text]
Artwork Changes

↓

Palette Blend

↓

Atmosphere Blend

↓

Material Update

Avoid.

[text]
Artwork Changes

↓

Immediate Colour Replacement

The environment should appear to breathe.

Not flash.

Contextual Synthesis

Different Contexts should produce different synthesis strategies.

Example.

Playback.

Minimal Atmosphere

Browsing.

Moderate Atmosphere

Reading.

Warm Quiet Atmosphere

Administration.

Neutral Environment

The same artwork should therefore influence different experiences differently.

Domain Synthesis

Future domains may influence atmospheric weighting.

Example.

Anime.

Artwork Influence

High

Music.

Artwork Influence

Medium

Administration.

Artwork Influence

Minimal

The platform should preserve behavioural consistency while respecting the emotional expectations of each domain.

Performance Strategy

Runtime Synthesis should prioritise:

- caching
- incremental updates
- palette reuse
- deferred recomputation

The synthesis engine should update primarily when:

- Hero changes
- Focus changes
- artwork changes

- theme changes

Ordinary interaction should not repeatedly regenerate atmosphere.

Failure Behaviour

If Runtime Synthesis cannot produce an atmosphere:

Preferred.

[text]
Neutral Theme

↓

Brand Identity

↓

Continue

Avoid.

[text]
No Artwork

↓

Broken Colours

↓

Visible Error

Atmosphere is an enhancement.

The platform should continue functioning normally without it.

Plugin Participation

Plugins may contribute:

- artwork
- metadata
- contextual information

Plugins never participate directly in synthesis.

The synthesis engine remains entirely platform owned.

This guarantees visual consistency regardless of extension source.

Good Examples

Film

Hero artwork.

↓

Cool environmental reflections.

↓

Neutral interface.

↓

Readable typography.

Users feel the atmosphere.

They do not consciously notice colour processing.

Book

Warm illustrated cover.

↓

Soft amber reflections.

↓

Paper-like acrylic.

↓

Quiet interface.

The experience feels literary rather than cinematic.

Music

Album artwork.

↓

Subtle ambient glow.

↓

Playback remains visually dominant.

↓

Controls remain semantically consistent.

The artwork influences mood.

Not interaction.

Anti-patterns

Dynamic Theme Replacement

Artwork completely replaces the current theme.

Brand identity disappears.

Material Bypass

Artwork colours applied directly to components.

Materials become irrelevant.

Atmosphere Everywhere

Every interface region reacts equally.

Visual noise increases.

Hierarchy weakens.

Instant Synthesis

Artwork changes produce immediate environmental changes.

Continuity disappears.

Runtime Synthesis Model

[mermaid]
flowchart TD

```
Artwork
Artwork --> PaletteAnalysis
PaletteAnalysis
PaletteAnalysis --> RuntimeAtmosphere
Theme
Theme --> RuntimeAtmosphere
Accessibility
Accessibility --> RuntimeAtmosphere
RuntimeAtmosphere
RuntimeAtmosphere --> MaterialSystem
MaterialSystem
MaterialSystem --> Presentation
```

The synthesis engine exists between semantic meaning and material expression.

It should never bypass either layer.

Relationship To Future Specifications

Future specifications are expected to define:

- palette clustering
- colour interpolation
- HDR-aware synthesis
- GPU pipelines

- UV-indexed refraction integration
- multi-display synchronisation
- ambient lighting adaptation

These systems should all implement the architectural model established here.

Summary

Runtime Synthesis is the process that gives Mosaic its distinctive visual character.

It does not create identity.

It does not create meaning.

It creates atmosphere.

By combining stable semantic design with subtle environmental adaptation, Mosaic becomes capable of reflecting the user's entertainment while remaining calm, recognisable and unmistakably itself.

Review Status

Status

Draft

Next File

11-governance.md

Colour System Governance

Purpose

The Colour System is one of the most recognisable aspects of the Mosaic Design System.

Unlike implementation details, colour establishes long-term visual memory.

Poor governance leads to:

- inconsistent branding
- semantic drift
- accessibility regressions
- visual fragmentation
- plugin inconsistency

This chapter defines how the Colour System should evolve while preserving a coherent visual language across every Mosaic client and extension.

Governance Philosophy

The Colour System should evolve continuously.

Its meaning should not.

The objective of governance is therefore not preserving individual colours.

It is preserving:

- semantic consistency
- accessibility
- emotional coherence
- brand identity

The interface may look different in ten years.

It should still feel unmistakably like Mosaic.

Colour Is Architecture

Within Mosaic, colour is treated as architecture rather than decoration.

Changing:

Surface.Hero

is not simply a visual decision.

It potentially affects:

- hierarchy

- accessibility
- runtime atmosphere
- materials
- components
- every supported client

Consequently, colour changes should be reviewed with the same care as API changes.

Ownership

The Colour System has clearly defined ownership.

Layer	Owner
Primitive Colours	Design Systems
Brand Palette	Design Systems
Semantic Colours	Design Systems
Runtime Atmosphere	Runtime Platform
Theme Resolution	Runtime Platform
Client Rendering	Client Teams

Ownership preserves architectural integrity.

It does not prevent contribution.

Stable Responsibilities

The following responsibilities should remain extremely stable.

- Brand identity
- Semantic colour names
- Accessibility priorities
- Theme architecture
- Colour hierarchy

These concepts form part of the public Design System API.

Breaking them should require architectural review.

Evolvable Responsibilities

The following may evolve more frequently.

- Primitive palette values
- Runtime atmosphere algorithms

- Artwork extraction heuristics
- Material rendering
- Platform implementation

Implementation should remain flexible.

Meaning should remain stable.

Introducing New Colours

Before introducing a new Semantic Colour, contributors should answer the following.

Question One

Does an existing Semantic Colour already communicate this meaning?

Question Two

Is this genuinely a new semantic responsibility...

or merely another visual treatment?

Question Three

Can Runtime Atmosphere solve this instead?

Question Four

Will this colour remain meaningful independently of the current visual style?

Question Five

Would another contributor naturally discover this token?

If uncertainty remains...

Refinement should continue before implementation.

Brand Governance

Brand Colours require the highest level of stability.

Changes should occur only when:

- Mosaic identity evolves
- accessibility requires adjustment
- branding strategy changes

Artwork should never influence Brand.

Brand belongs exclusively to Mosaic.

Semantic Governance

Semantic Colours represent the public language of the Design System.

Examples.

`Surface.Primary`

`Text.Secondary`

`Action.Primary`

These names should remain stable across:

- redesigns
- themes
- runtime improvements
- client platforms

Semantic drift should be considered architectural debt.

Runtime Governance

Runtime Atmosphere should evolve independently.

Examples include:

- improved palette extraction
- better blending
- GPU acceleration
- HDR support
- smarter adaptation

These improvements should require no changes to consuming applications.

Accessibility Governance

Accessibility possesses higher authority than aesthetics.

No visual proposal should be accepted if it weakens:

- readability
- semantic clarity
- colour independence
- contrast

Runtime Atmosphere should always adapt to accessibility.

Accessibility should never adapt to atmosphere.

Theme Governance

Themes should:

- reinterpret implementation
- preserve semantic meaning

Themes should never:

- introduce new hierarchy
- redefine interaction
- alter composition

Those concepts belong to MDL.

Plugin Governance

Extensions should never introduce:

- Brand colours
- Runtime colour systems
- Independent themes
- Alternative semantic colour hierarchies

Plugins consume the Colour System.

They do not extend its architecture.

This guarantees one coherent visual identity.

Review Questions

Every colour proposal should answer:

- Does this strengthen semantic clarity?
- Does this improve accessibility?
- Does this preserve Brand identity?
- Does Runtime already solve this?
- Will this still make sense after a redesign?
- Would removing this colour reduce understanding?

If the answer to the final question is "no"...

The colour probably does not belong.

Colour Drift

Colour Drift occurs when:

- duplicate semantic colours appear
- branding leaks into semantics
- atmosphere replaces hierarchy
- plugins introduce independent palettes
- components consume primitive colours

Colour Drift weakens the Design System gradually.

It should be corrected as early as possible.

Governance Workflow

[mermaid]
flowchart TD

```
Proposal --> SemanticReview["Semantic Review"]
SemanticReview["Semantic Review"] --> ExistingMeaning["Existing Meaning?"]
ExistingMeaning["Existing Meaning?"] -->|Yes| Reuse
ExistingMeaning["Existing Meaning?"] -->|No| ArchitectureReview["Architecture Review"]
ArchitectureReview["Architecture Review"] --> AccessibilityReview["Accessibility Review"]
AccessibilityReview["Accessibility Review"] --> RuntimeReview["Runtime Review"]
RuntimeReview["Runtime Review"] --> Approval
Approval --> Implementation
```

Reuse should always be preferred over expansion.

Validation

Future tooling should automatically validate:

- duplicate semantic colours
- contrast compliance
- unresolved tokens
- primitive colour consumption
- runtime compatibility
- theme completeness

Automated validation should reinforce architectural review.

Not replace it.

Success Criteria

The Colour System succeeds when:

- Brand remains instantly recognisable.
- Artwork enriches rather than dominates.
- Accessibility remains uncompromised.
- Runtime adaptation feels natural.
- Contributors naturally reuse Semantic Colours.
- Extensions visually disappear into the platform.

The strongest Colour System is one users rarely think about consciously.

They simply feel that everything belongs together.

Architectural Decisions

ADR	Decision
ADR-093	Semantic colour meaning is treated as a long-lived architectural contract.
ADR-094	Accessibility always has higher authority than runtime atmosphere.
ADR-095	Brand identity is independent from entertainment atmosphere.
ADR-096	Plugins consume the Colour System but never redefine it.

Review Status

Status

Draft

Next File

12-adrs.md

Architectural Decision Records

Purpose

The Architectural Decision Records (ADRs) contained within MDS-002 preserve the reasoning behind the Mosaic Colour System.

Where previous specifications established:

- Vision
- Principles
- Mental Model
- Interaction
- Composition
- Token Architecture

MDS-002 establishes the visual language through which those concepts become emotionally expressive.

These ADRs explain why the Colour System deliberately separates:

- Brand
- Semantic Colour
- Runtime Atmosphere
- Themes
- Accessibility

Future contributors should understand these decisions before proposing changes to the visual identity of Mosaic.

ADR Format

Every Mosaic ADR follows the same structure.

[text]
ADR Number

Status

Context

Decision

Consequences

Alternatives Considered

Related Specifications

Each ADR should record one significant architectural decision.

ADR-097

Title

Separate Brand From Atmosphere

Status

Accepted

Context

Many media applications allow artwork to dominate their visual identity.

Founder workshops consistently identified the need for a recognisable Mosaic identity that remained independent from currently playing media.

Decision

Brand Colours remain stable.

Runtime Atmosphere adapts.

The two systems remain architecturally independent.

Consequences

Users continue recognising Mosaic regardless of current entertainment.

ADR-098

Title

Artwork Influences Environment Rather Than Interface

Status

Accepted

Context

Direct artwork recolouring creates visually unstable interfaces.

Decision

Artwork contributes Runtime Atmosphere rather than direct component colours.

Atmosphere influences Materials.

Materials influence Presentation.

Consequences

Entertainment emotionally influences the interface without replacing the Design System.

ADR-099

Title

Semantic Colours Are Stable

Status

Accepted

Context

Colour values are expected to evolve significantly over time.

Semantic meaning should not.

Decision

Applications consume Semantic Colours rather than Primitive Colours.

Primitive values remain implementation details.

Consequences

Future redesigns become dramatically less disruptive.

ADR-100

Title

Accessibility Has Higher Authority Than Atmosphere

Status

Accepted

Context

Artwork frequently produces colour combinations unsuitable for accessible interfaces.

Decision

Accessibility validation occurs before Runtime Atmosphere reaches Presentation.

Consequences

Immersion never compromises readability.

ADR-101

Title

Themes Interpret Rather Than Redefine

Status

Accepted

Context

Maintaining separate Light and Dark design systems fragments conceptual consistency.

Decision

Themes become visual interpretations of identical semantic architecture.

Consequences

Users experience one Design Language regardless of active theme.

ADR-102

Title

Runtime Colour Is Deterministic

Status

Accepted

Context

Adaptive colour systems frequently become visually unpredictable.

Decision

Given identical runtime inputs, the Runtime Resolver must always produce identical colour outputs.

Consequences

Caching, testing and cross-platform consistency become significantly easier.

ADR-103

Title

Atmosphere Is Material-Led

Status

Accepted

Context

Early exploration applied artwork colours directly to interface surfaces.

This weakened hierarchy and obscured brand identity.

Decision

Atmosphere should primarily influence Materials rather than component colours.

Consequences

Future acrylic and refraction systems become the primary visual expression of atmosphere.

ADR-104

Title

Extensions Consume Rather Than Define Colour

Status

Accepted

Context

Allowing extensions to introduce independent colour systems fragments visual identity.

Decision

Extensions consume:

- Semantic Colours
- Runtime Atmosphere

The platform remains solely responsible for colour generation.

Consequences

Community extensions inherit future visual improvements automatically.

ADR-105

Title

Colour Exists To Support Understanding

Status

Accepted

Context

Many modern interfaces use colour primarily as decoration.

Founder workshops consistently reinforced that hierarchy and understanding should exist independently from colour.

Decision

Colour reinforces meaning.

It never becomes the only mechanism communicating meaning.

Consequences

Accessibility improves.

The interface remains understandable even with greatly reduced colour information.

ADR Relationships

```
[mermaid]
flowchart TD
    ADR097["ADR097["Brand"]"]
    ADR098["ADR098["Artwork"]"]
    ADR099["ADR099["Semantic"]"]
    ADR100["ADR100["Accessibility"]"]
    ADR101["ADR101["Themes"]"]
    ADR102["ADR102["Runtime"]"]
    ADR103["ADR103["Materials"]"]
    ADR104["ADR104["Extensions"]"]
    ADR105["ADR105["Understanding"]"]

    ADR097 --> ADR099
    ADR099 --> ADR101
    ADR101 --> ADR102
    ADR098 --> ADR103
    ADR103 --> ADR102
    ADR100 --> ADR102
    ADR102 --> ADR104
    ADR105 --> ADR099
```

Together these decisions establish the architectural separation between:

- identity
- meaning
- emotion
- implementation

which defines the Mosaic Colour System.

Future ADRs

Future Colour System ADRs are expected to formalise:

- HDR Colour Pipeline
- Wide Gamut Displays

- Dynamic Contrast Balancing
- Multi-Monitor Colour Synchronisation
- Refraction Colour Transport
- Ambient Lighting Integration
- AI-assisted Artwork Analysis
- Material Spectral Rendering

These intentionally remain outside the scope of MDS-002 Version 0.1.

ADR Governance

Colour ADRs should change only when:

- accessibility research identifies deficiencies,
- semantic ambiguity exists,
- runtime architecture evolves,
- the Design Language itself changes.

Presentation trends should never justify architectural colour changes.

Summary

The ADRs contained within MDS-002 define the architectural identity of the Mosaic Colour System.

Rather than treating colour as decoration, Mosaic treats colour as:

- identity,
- meaning,
- atmosphere.

Each responsibility remains independent.

Together they create a colour system capable of evolving for many years while remaining recognisably Mosaic.

Review Status

Status

Draft

Next File

13-contributor-guidance.md

Contributor Guidance

Purpose

The Colour System is one of the most frequently consumed parts of the Mosaic Design System.

Every contributor, regardless of discipline, will eventually make colour-related decisions.

This chapter provides practical guidance for ensuring those decisions reinforce the architectural principles established throughout MDL and MDS.

The objective is not to make contributors think about colours.

It is to make them think about meaning.

Think In Meaning

Before selecting any colour ask:

What does this colour need to communicate?

Never begin with:

- cyan
- purple
- blue
- grey

Instead begin with:

- Hero
- Action
- Surface
- Status
- Atmosphere

Meaning always precedes colour.

Consume Semantic Colours

Applications should almost never consume Primitive Colours directly.

Preferred.

[text]
Action.Primary

Avoid.

[text]
Primitive.Colour.Cyan.500

The Semantic layer protects applications from future redesigns.

Always consume the highest semantic layer available.

Let Atmosphere Do Its Job

Contributors should never manually tint components to match artwork.

Incorrect.

[text]
Poster

↓

Tint Button

Preferred.

[text]
Poster

↓

Runtime Atmosphere

↓

Material

↓

Button

The Colour System already understands how atmosphere should propagate.

Application code should not duplicate that logic.

Preserve The Brand

Brand colours should be treated with restraint.

Ask:

Is this communicating Mosaic...

Or...

Is this merely decorating the interface?

If the answer is decoration...

The Brand Colour probably does not belong there.

Brand gains strength through consistency rather than frequency.

Think In Atmosphere

Artwork should influence mood.

Not identity.

Good question.

What kind of light would this artwork cast into the room?

Poor question.

What colour should this interface become?

The first produces Mosaic.

The second produces theme replacement.

Accessibility First

Whenever colour decisions conflict with readability:

Accessibility wins.

Always.

Examples.

Poor.

Beautiful

↓

Unreadable

Preferred.

Readable

↓

Beautiful

A visually impressive interface that weakens understanding is considered a design failure.

Avoid Component Colours

Components should never invent colours.

Example.

Poor.

[text]
Button

↓

Choose Blue

Preferred.

[text]
Button

↓

Action.Primary

The Design System owns colour.

Components communicate behaviour.

Runtime Is Platform-Owned

Applications and extensions should never:

- analyse artwork
- generate atmosphere
- resolve palettes
- blend themes

These responsibilities belong exclusively to the Runtime Colour System.

The platform guarantees consistency.

Applications simply consume the results.

Extensions

Extension authors should consume:

- Semantic Colours
- Runtime Tokens
- Composition Tokens

They should never introduce:

- brand palettes
- custom themes
- independent atmosphere
- alternative semantic colours

The extension should disappear into Mosaic rather than advertising itself.

Think About Light

One useful mental model for contributors is to imagine that colour represents light rather than paint.

Brand.



Identity.

Atmosphere.



Environmental light.

Materials.



Physical response.

Components.



Objects inside that environment.

Thinking this way naturally produces calmer interfaces.

Common Mistakes

Avoid the following.

Primitive Consumption

Directly consuming raw colour values.

Artwork Sampling

Applying extracted colours directly to components.

Brand Saturation

Using Brand Colours everywhere.

Decorative Colour

Adding colour because empty space feels uncomfortable.

Theme Logic

Embedding Light/Dark decisions directly into application code.

The Theme Resolver already owns those responsibilities.

Accessibility Overrides

Reducing readability for stronger atmosphere.

Accessibility always possesses higher authority.

Colour Review Questions

Before implementing any colour-related change ask:

- What meaning does this colour communicate?

- Could an existing Semantic Colour already solve this?
- Does Runtime already provide this behaviour?
- Will this remain meaningful after a redesign?
- Does this strengthen understanding?
- Does this compete with the entertainment?

If uncertainty remains...

The colour decision probably belongs inside the Design System rather than application code.

Colour Checklist

Every colour implementation should satisfy the following.

- ☐ Uses Semantic Colours rather than Primitive values.
- ☐ Preserves accessibility.
- ☐ Supports Runtime Atmosphere.
- ☐ Preserves Brand identity.
- ☐ Avoids component-specific colour logic.
- ☐ Remains implementation independent.
- ☐ Works across Light and Dark themes.
- ☐ Supports future runtime adaptation.

Final Guidance

The strongest Colour System is one contributors eventually stop thinking about.

Instead of asking:

"Which colour should this be?"

contributors should instinctively ask:

"What does this need to communicate?"

When that question becomes habitual, implementation naturally begins to align with the Mosaic Design Language.

The Colour System then ceases to be a palette.

It becomes a language.

Review Status

Status

Draft

Next File

glossary.md

Glossary

Purpose

This glossary defines the terminology introduced by **MDS-002 — Colour System**.

Unlike previous glossaries, these definitions describe architectural colour concepts rather than implementation values.

Every future MDS specification should use these definitions consistently.

Changing colour terminology changes how contributors reason about the Design System.

For that reason, these definitions should remain stable.

A

Accessibility Colour

A resolved colour that satisfies accessibility requirements while preserving semantic meaning.

Accessibility Colours are generated through Theme Resolution rather than manually authored.

Atmosphere

The adaptive environmental lighting generated from the user's current entertainment.

Atmosphere is derived from artwork.

It influences Materials.

It never replaces Brand or Semantic Colours.

Atmosphere Token

A Runtime Token representing environmental colour characteristics.

Examples include:

- Atmosphere.Primary
- Atmosphere.Reflection
- Atmosphere.Highlight

Atmosphere Tokens are generated.

They are never authored manually.

B

Brand Colour

A stable colour representing the Mosaic identity.

Brand Colours communicate:

- recognition
- consistency
- product identity

They intentionally do not communicate runtime emotion.

C

Colour Resolution

The deterministic process that transforms Semantic Colour intent into resolved implementation values.

Colour Resolution considers:

- Theme
- Accessibility
- Runtime Atmosphere
- Device

before producing final colours.

Colour Synthesis

The runtime process responsible for generating environmental atmosphere from entertainment artwork.

Colour Synthesis influences Materials.

It never changes semantic meaning.

L

Light Source

The conceptual origin of Runtime Atmosphere.

Within Mosaic, artwork is treated as a light source rather than a colour palette.

This distinction underpins the Refraction System.

M

Material Colour

The colour produced after Runtime Atmosphere has interacted with a Material.

Material Colours are generated.

They are not authored.

P

Primitive Colour

A measurable physical colour value.

Examples include:

- Cyan 500
- Slate 950
- White
- Black

Primitive Colours intentionally contain no semantic meaning.

R

Reflected Colour

A colour generated by the interaction between Runtime Atmosphere and Materials.

Reflected Colours should appear subtle and environmental.

They should never dominate semantic hierarchy.

Runtime Atmosphere

The adaptive environmental colour model generated from the user's current World.

Runtime Atmosphere represents emotional lighting rather than interface colouring.

S

Semantic Colour

A colour role representing meaning rather than implementation.

Examples include:

- Surface.Primary
- Text.Secondary
- Action.Primary
- Status.Success

Applications should consume Semantic Colours rather than Primitive Colours.

Surface Colour

A Semantic Colour describing structural hierarchy.

Surface Colours communicate:

- depth
- grouping
- organisation

They should not communicate interaction or status.

T

Theme

A deterministic interpretation of Semantic Colours.

Themes implement:

- Light
- Dark
- OLED
- Accessibility

without changing semantic meaning.

Theme Resolution

The process by which Semantic Colours become theme-specific implementation values.

Theme Resolution occurs before Runtime Atmosphere.

Cross References

Specification	Primary Concepts
MDL-001 Vision	Companion, Immersion
MDL-002 Principles	Content Leads
MDL-003 Mental Model	World, Focus
MDL-004 Interaction Model	Runtime Behaviour
MDL-005 Composition Model	Hierarchy, Hero
MDS-001 Design Token Architecture	Semantic Tokens, Runtime Tokens
MDS-003 Material System	Acrylic, Refraction

Terminology Rules

Future contributors should:

- describe meaning before colour values
- describe atmosphere before implementation
- distinguish Brand from Atmosphere
- distinguish Semantic Colour from Primitive Colour
- avoid implementation-specific terminology inside architectural documentation

Colour terminology should remain independent from rendering technology.

Review Status

Status

Draft

Next File

references.md

References

Purpose

This document records the architectural influences, design philosophies and implementation concepts that informed **MDS-002 — Colour System**.

Unlike implementation documentation, these references exist to explain the reasoning behind the Colour System rather than prescribe specific algorithms.

The Mosaic Colour System intentionally combines ideas from:

- visual design
- perception
- accessibility
- material design
- lighting
- runtime adaptation

into a single architectural model centred around entertainment rather than software.

Reading Order

Contributors should approach references in the following order.

1. MDL Specifications 2. MDS Token Architecture 3. Colour System 4. Material System 5. Runtime Systems 6. Platform Implementations

The Mosaic Design Language remains the primary authority.

External references provide context.

Internal References

MDL-001 — Vision

Provides:

- Product identity
- Companion philosophy
- Immersion
- Entertainment-first thinking

Every colour decision should ultimately reinforce the experience described within the Vision.

MDL-002 — Principles

Provides:

- Content Leads
- Context Before Prediction
- Be A Companion
- Movement Preserves Understanding

The Colour System should strengthen these principles.

It should never replace them.

MDL-003 — Mental Model

Provides:

- World
- Focus
- Context
- Information

Runtime Atmosphere intentionally reflects the user's current World rather than arbitrary interface state.

MDL-004 — Interaction Model

Provides:

- Behaviour
- Continuity
- Runtime evolution

Atmosphere should evolve with interaction rather than behaving like independent theme switching.

MDL-005 — Composition Model

Provides:

- Hero
- Hierarchy
- Priority
- Density

Colour should reinforce Composition.

It should never redefine it.

MDS-001 — Design Token Architecture

Provides:

- Primitive Tokens
- Semantic Tokens
- Runtime Tokens
- Resolution Pipeline

The Colour System implements this architecture.

It should not modify it.

Future Specifications

The following specifications depend directly upon MDS-002.

- MDS-003 Material System
- MDS-004 Typography
- MDS-005 Motion System
- MDS-006 Composition Engine
- MDS-007 Tile Framework
- MDS-008 Component Library

These specifications consume Semantic Colours and Runtime Atmosphere rather than creating independent colour systems.

Colour Theory

The Mosaic Colour System intentionally distinguishes between:

- identity
- meaning
- emotion

Traditional colour theory often merges these responsibilities.

Mosaic deliberately separates them into:

Brand

↓

Semantic

↓

Atmosphere

This separation allows each system to evolve independently.

Human Perception

The Colour System is influenced by several characteristics of human perception.

Examples include:

- simultaneous contrast
- colour adaptation
- luminance perception
- visual hierarchy
- peripheral awareness

Rather than using colour to attract attention, Mosaic primarily uses colour to support understanding and atmosphere.

Environmental Lighting

One of the defining influences behind the Mosaic Colour System is environmental lighting.

Rather than treating artwork as a palette...

Mosaic treats artwork as a light source.

This philosophical distinction influences:

- Runtime Atmosphere
- Material System
- Refraction Engine
- Acrylic rendering

It is one of the defining characteristics of the Mosaic visual identity.

Accessibility

Accessibility is considered a first-class architectural concern.

The Colour System intentionally assumes:

Accessibility

↓

Semantic Meaning

↓

Atmosphere

↓

Presentation

This ordering ensures that runtime adaptation never compromises readability.

Future implementations should validate accessibility continuously rather than treating it as a final review step.

Runtime Design

Unlike traditional Design Systems, Mosaic assumes colour changes over time.

Examples include:

- artwork
- Focus
- Context
- playback
- accessibility
- user preference

The Colour System therefore treats runtime adaptation as an architectural responsibility rather than an implementation feature.

Material Interaction

The Colour System intentionally stops at Materials.

Conceptually.

[text]
Colour

↓

Atmosphere

↓

Materials

↓

Presentation

Materials determine:

- translucency
- diffusion
- glow
- refraction

Colour provides environmental influence.

This separation allows future Material Systems to evolve independently.

Platform Independence

The Colour System intentionally avoids platform-specific implementation.

Every client should consume identical semantic meaning while generating implementation appropriate for:

- Web
- Flutter
- SwiftUI
- Compose
- Future platforms

The visual language should remain recognisable regardless of rendering technology.

Mosaic-Specific Influences

The Colour System emerged directly from the architectural decisions made during Mosaic discovery.

Significant ideas include:

- artwork should illuminate rather than recolour the interface
- runtime atmosphere should remain subtle
- acrylic should behave like illuminated material rather than frosted glass
- colour should reinforce composition rather than replace it
- brand identity should remain recognisable regardless of current entertainment

These concepts distinguish Mosaic from existing entertainment platforms.

Relationship To The Refraction System

The Runtime Atmosphere defined within MDS-002 intentionally provides one of the primary inputs into the Mosaic Refraction System.

Conceptually.

[text]
Artwork

↓

Colour Extraction

↓

Runtime Atmosphere

↓

Material Refraction

↓

Presentation

This relationship will be formalised within the Material System specification.

The Colour System intentionally stops before physical rendering begins.

Normative References

Required reading before contributing to MDS-002.

- MDL-001 Vision
- MDL-002 Principles
- MDL-003 Mental Model
- MDL-004 Interaction Model
- MDL-005 Composition Model
- MDS-001 Design Token Architecture

Together these specifications define the conceptual foundation of the Colour System.

Informative References

Future contributors may also wish to review:

- MDS-003 Material System
- MDS-005 Motion System
- MDS-006 Composition Engine
- MDS-008 Component Library

These specifications describe how colour becomes a rendered experience.

Living Document

This reference list should remain intentionally concise.

References should only be introduced when they materially influence:

- semantic architecture
- runtime atmosphere
- accessibility
- implementation boundaries

The purpose of this document is to preserve architectural reasoning rather than become an exhaustive bibliography.

Completion

This concludes **MDS-002 — Colour System**.

The next specification in the Mosaic Design System is:

MDS-003 — Material System

Where MDS-002 defines **how colour communicates meaning**, MDS-003 defines **how those colours physically exist within the Mosaic interface**.

It formalises:

- Acrylic
- Refraction
- Light transport
- UV-indexed atmosphere
- Material hierarchy
- Canvas
- Hero surfaces
- Overlay materials
- Adaptive translucency

This specification is expected to become one of the defining technical and visual documents of the entire Mosaic platform.